

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Sultan Syarief Usman - 5024231006

2026

1 Langkah-Langkah Percobaan

Percobaan 1:

- Menyiapkan board STM32 Blackpill, programmer ST-Link V2, kabel jumper, dan kabel USB.
- Menyambungkan pin SWDIO, SWCLK, GND, dan 3.3V dari ST-Link ke Blackpill menggunakan kabel jumper, lalu mencolokkan ST-Link ke laptop.
- Mengatur konfigurasi platformio.ini untuk board blackpill-f401cc dengan framework Arduino dan protokol upload stlink.
- Memasukkan kode program blink (LED bawaan PC13) ke file src/main.cpp, lalu melakukan build dan upload hingga LED berkedip setiap setengah detik.

Percobaan 2:

- Menyiapkan board ESP32 DevKit dan mencolokkannya langsung ke laptop menggunakan kabel USB hingga port terdeteksi.
- Mengubah konfigurasi platformio.ini menjadi board esp32dev dengan framework espidf, serta mengganti file utama dari main.cpp menjadi src/main.c.
- Mengetikkan kode program blink untuk LED onboard (GPIO 2) ke dalam src/main.c, lalu menjalankan build dan upload hingga LED berkedip.

Percobaan 3:

- Menyiapkan ESP32 DevKit, breadboard, dua pushbutton, dua resistor 10k, dan kabel jumper male-male.
- Merangkai konfigurasi eksternal dengan menghubungkan tombol pertama ke GPIO 32 (ke 3.3V via resistor 10k dan kaki lainnya ke GND) serta tombol kedua ke GPIO 33 (ke GND via resistor 10k dan kaki seberangnya ke 3.3V).
- Mengatur kembali platformio.ini ke framework Arduino dan membuat ulang file src/main.cpp.
- Memasukkan kode pembacaan eksternal ke src/main.cpp, melakukan upload, dan mengecek hasil pembacaan tombol di Serial Monitor.
- Mengubah rangkaian ke konfigurasi internal dengan mencabut kedua resistor 10k dari breadboard tanpa mengubah posisi kabel tombol lainnya.
- Mengganti isi src/main.cpp dengan program pull-up dan pull-down internal, lalu melakukan flash ulang untuk membandingkan hasilnya di Serial Monitor.

Percobaan 4:

- Menyiapkan ESP32 DevKit, breadboard, satu pushbutton, dan kabel jumper.
- Memasang satu kaki pushbutton ke pin GPIO 25 dan kaki pasangannya langsung ke jalur GND untuk memanfaatkan fitur pull-up internal.

- Memasukkan kode program penghitung debounce ke dalam file src/main.cpp menggunakan pengaturan platformio.ini yang sama seperti percobaan sebelumnya.
- Melakukan upload program dan menguji tekanan tombol secara cepat maupun lambat, sembari mengubah nilai DEBOUNCE-DELAY (50 ms, 10 ms, dan 200 ms) lalu mem-flash ulang untuk mengamati perbandingan angkanya di Serial Monitor.

Percobaan 5:

- Menyiapkan ESP32 DevKit, breadboard, satu sensor cahaya LDR, satu resistor 10k, dan kabel jumper.
- Merangkai pembagi tegangan dengan menghubungkan satu kaki LDR ke 3.3V, sedangkan kaki lainnya disambungkan ke GPIO 34 sekaligus diteruskan ke GND melalui resistor 10k.
- Memasukkan kode pembacaan ADC ke src/main.cpp dengan konfigurasi platformio.ini yang tetap berada di framework Arduino.
- Melakukan build dan upload program ke board, lalu memantau perubahan nilai analog dan tegangan di Serial Monitor saat LDR ditutup tangan maupun disinari cahaya.

2 Analisis Hasil Percobaan

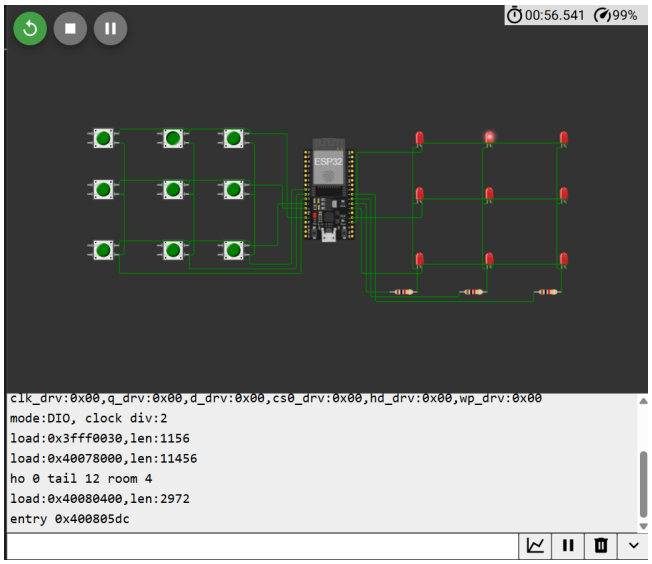
Pada percobaan pertama, pengujian dilakukan untuk mencoba flashing program blink ke board STM32 Blackpill lewat ST-Link dengan framework Arduino. Hasil yang diharapkan adalah LED bawaan di pin PC13 bisa nyala dan mati bergantian setiap 500 ms, dan hasil praktikumnya sudah sesuai. Hal ini menunjukkan kalau koneksi jalur SWD dari ST-Link sudah pas.
Pada percobaan kedua, tujuannya adalah mencoba program blink di ESP32 menggunakan framework native ESP-IDF. Hasil yang diharapkan yaitu LED onboard di GPIO 2 bisa berkedip setiap setengah detik, dan hasilnya juga berjalan sesuai harapan. Dari percobaan ini terlihat kalau ESP32 bisa langsung di-flash lewat kabel USB tanpa alat tambahan, serta membuktikan kalau struktur program berbasis FreeRTOS, bisa menjalankan kontrol GPIO dasar sama baiknya seperti setup dan loop di Arduino.
Pada percobaan ketiga, kita membandingkan cara kerja resistor pull-up (GPIO 32) dan pull-down (GPIO 33) pada tombol, baik yang dirangkai secara eksternal maupun yang diaktifkan secara internal lewat program. Hasil yang diharapkan adalah kedua konfigurasi tersebut menampilkan pembacaan yang sama persis di Serial Monitor, dan saat dicoba hasilnya memang sesuai. Ini membuktikan kalau resistor internal bawaan ESP32 punya fungsi yang sama dengan resistor fisik 10k untuk menahan nilai pin supaya tidak floating (acak), di mana pull-up terbaca LOW saat ditekan dan pull-down terbaca HIGH saat ditekan.
Pada percobaan keempat, pengujian fokus untuk melihat efek getaran kontak mekanik (bouncing) saat tombol ditekan dan cara mengatasinya pakai teknik debouncing software di GPIO 25. Hasil yang diharapkan adalah penghitung dengan debounce cuma bertambah 1 tiap kali tombol ditekan, sedangkan penghitung tanpa debounce bisa loncat lebih dari 1, dan hasil percobaannya sesuai. Selain itu, percobaan saat mengubah nilai delay (10 ms, 50 ms, dan 200 ms) menunjukkan kalau waktu tunggu yang terlalu cepat (10 ms) masih rentan bocor terkena getaran, sedangkan kalau terlalu lama (200 ms) bikin tombol jadi kurang responsif waktu ditekan cepat, sehingga 50 ms jadi nilai yang paling

pas.

Pada percobaan kelima, kita mencoba membaca input analog (ADC) di GPIO 34 menggunakan rangkaian pembagi tegangan dari sensor cahaya LDR dan resistor 10k. Harapannya, nilai ADC (rentang 0–4095) dan tegangan di Serial Monitor bakal berubah-ubah mengikuti terang-gelapnya cahaya saat LDR ditutup tangan atau disinari lampu, tapi hasil praktikum kemarin ternyata tidak sesuai karena nilainya stagnan (tidak berubah). Padahal kalau dicek rangkaiannya sudah benar, jadi kemungkinan penyebabnya adalah kaki komponen LDR atau kabel jumper yang kurang nancap sempurna di lubang breadboard yang longgar sehingga tegangannya tidak masuk ke pin, atau bisa juga karena komponen LDR yang dipakai memang sudah rusak dari awal sehingga nilai resistansinya tidak berubah saat terkena cahaya.

3 Hasil Tugas Modul

3.1 Simulasi Rangkaian



Gambar 1: Hasil Rangkaian Tugas Modul

3.2 Source code yang digunakan

```
const int btnRows[3] = {13, 12, 14};
const int btnCols[3] = {27, 26, 25};
const int ledRows[3] = {15, 2, 4};
const int ledCols[3] = {16, 17, 5};

bool btnState[3][3] = {false};
bool lastRawState[3][3] = {false};
unsigned long lastDebounceTime[3][3] = {0};
const unsigned long debounceDelay = 10; // Waktu stabilisasi dalam milidetik

void setup() {
  for (int i = 0; i < 3; i++) {
    pinMode(btnRows[i], OUTPUT);
    digitalWrite(btnRows[i], HIGH);
```

```
    pinMode(btnCols[i], INPUT_PULLUP);
  }

  for (int i = 0; i < 3; i++) {
    pinMode(ledRows[i], OUTPUT);
    digitalWrite(ledRows[i], LOW);
    pinMode(ledCols[i], OUTPUT);
    digitalWrite(ledCols[i], HIGH);
  }
}

void scanButtons() {
  for (int r = 0; r < 3; r++) {
    digitalWrite(btnRows[r], LOW);
    delayMicroseconds(10);

    for (int c = 0; c < 3; c++) {
      bool rawState = (digitalRead(btnCols[c]) == LOW);

      if (rawState != lastRawState[r][c]) {
        lastDebounceTime[r][c] = millis();
      }

      if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
        if (rawState != btnState[r][c]) {
          btnState[r][c] = rawState;
        }
      }

      lastRawState[r][c] = rawState;
    }

    digitalWrite(btnRows[r], HIGH);
  }
}

void scanLEDs() {
  for (int r = 0; r < 3; r++) {
    for (int i = 0; i < 3; i++) {
      digitalWrite(ledRows[i], LOW);
    }

    for (int c = 0; c < 3; c++) {
      if (btnState[r][c] == true) {
        digitalWrite(ledCols[c], LOW);
      } else {
        digitalWrite(ledCols[c], HIGH);
      }
    }

    digitalWrite(ledRows[r], HIGH);

    delay(2);
  }
```

```
69 }
70
71 void loop() {
72   scanButtons();
73   scanLEDs();
74 }
```

3.3 Pertanyaan Analisis

- Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing?
  Saat tombol ditekan, pelat logam di dalamnya bergetar (bouncing) sehingga memicu sinyal putus-nyambung yang sangat cepat. Tanpa debouncing, satu tekanan saja bisa terbaca berkali-kali oleh mikrokontroler dan bikin nyala-mati LED jadi kacau atau berkedip sendiri.

- Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan kolom tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?
  Jika memakai pull-up, pin kolom bernilai HIGH saat tombol tidak ditekan (atau LOW jika memakai pull-down). Pin tidak boleh dibiarkan mengambang (floating) karena mudah menangkap gangguan listrik (noise) yang bikin status tombol terbaca acak dan LED menyala sendiri tanpa ditekan.

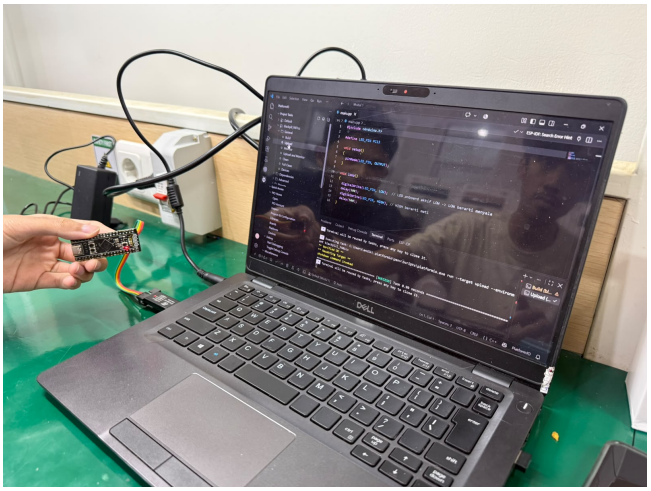
- Andaikan salah satu Tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?
  Digital input hanya mengenal dua kondisi mutlak (HIGH atau LOW / nyala atau mati), sedangkan analog input melalui ADC membaca perubahan tegangan secara bertahap (kontinu). Pada ESP32 (resolusi 12-bit), tegangan 0–3.3V diubah menjadi rentang angka 0–4095, sehingga bisa mendeteksi berbagai tingkat kecerahan cahaya pada LDR.

4 Kesimpulan

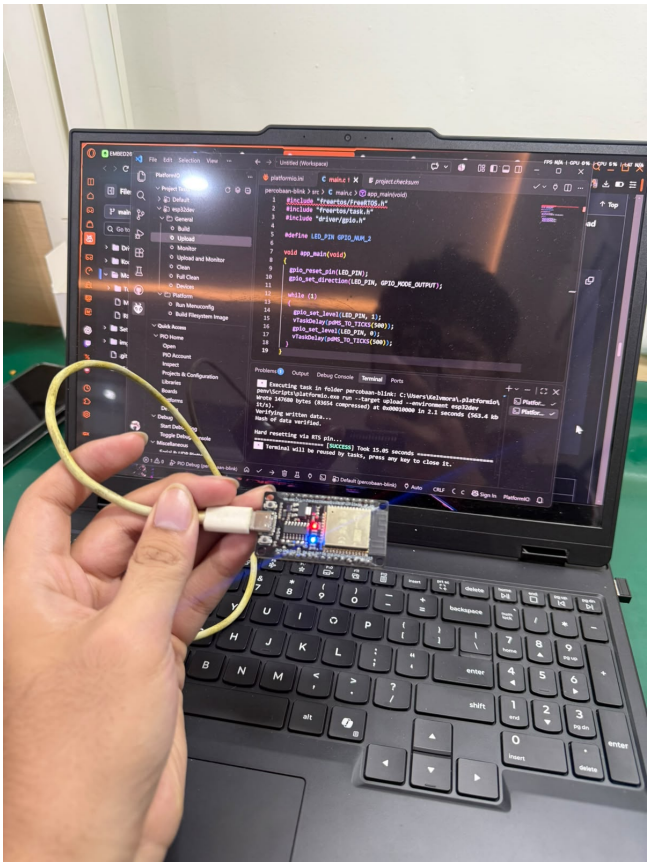
Dari seluruh percobaan pada praktikum Modul 1 Pengantar Mikrokontroler ini, dapat disimpulkan bahwa PlatformIO memudahkan pemrograman lintas board dan framework, baik saat melakukan flashing STM32 Blackpill lewat ST-Link dengan Arduino maupun ESP32 langsung via USB dengan ESP-IDF. Pada pengujian input digital di ESP32, penggunaan resistor pull-up dan pull-down internal terbukti mampu menggantikan peran resistor fisik 10k dengan hasil pembacaan yang sama persis untuk mencegah kondisi floating pada pin. Selain itu, penerapan debouncing berbasis software dengan waktu tunggu 50 ms terbukti paling pas untuk meredam gangguan getaran kontak mekanik (bouncing) tanpa membuat respons tombol menjadi lambat. Sementara itu, pada pengujian input analog (ADC) menggunakan sensor LDR, nilai pembacaan di Serial Monitor yang tidak berubah menunjukkan bahwa keberhasilan pembacaan tegangan analog sangat bergantung pada kondisi fisik sensor serta kerapatan koneksi kaki komponen pada breadboard.

5 Lampiran

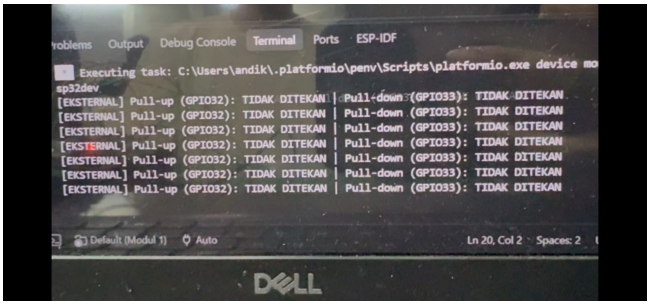
5.1 Dokumentasi saat praktikum



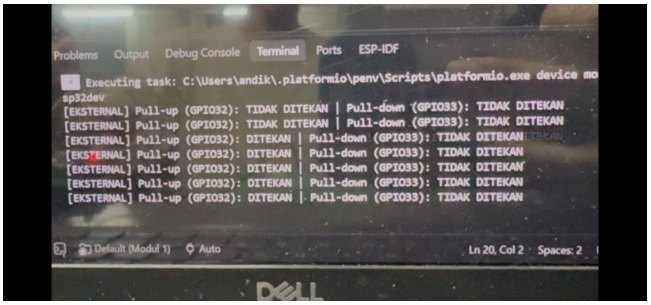
Gambar 2: Percobaan 1.



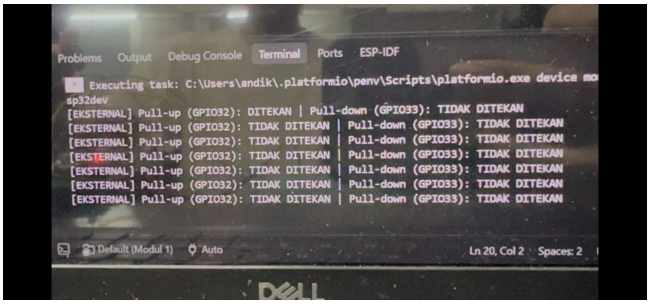
Gambar 3: Percobaan 2.



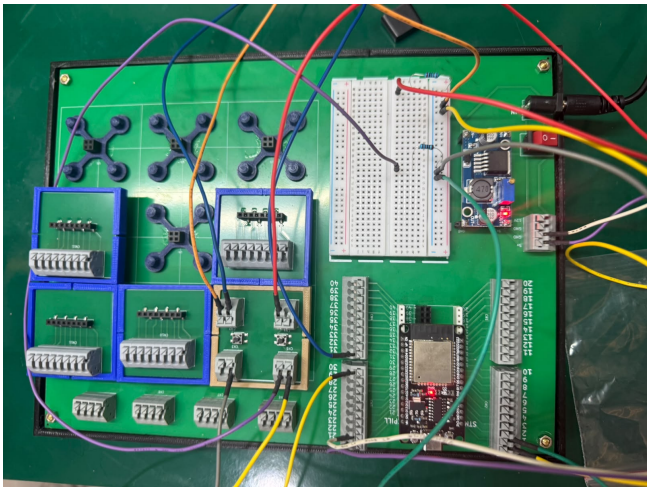
Gambar 4: Percobaan 3a - State 0.



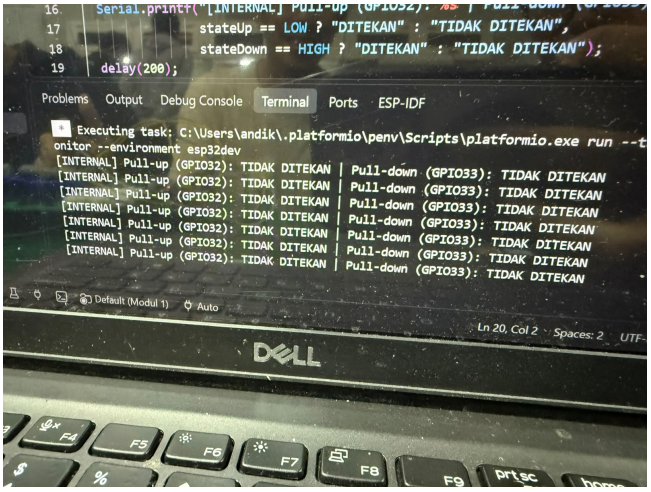
Gambar 5: Percobaan 3a - State 1.



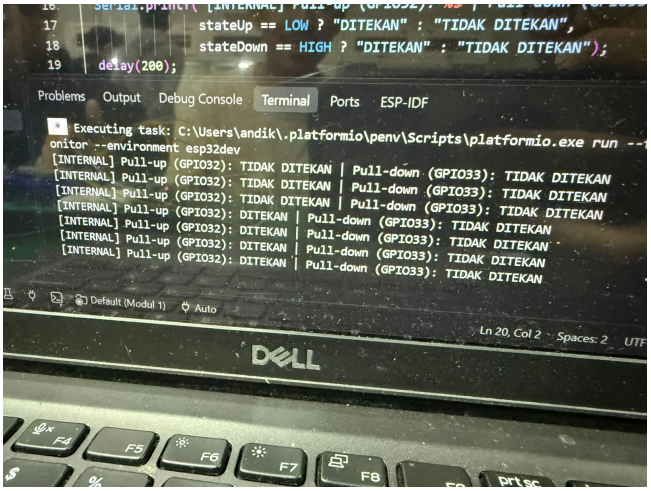
Gambar 6: Percobaan 3a - State 3.



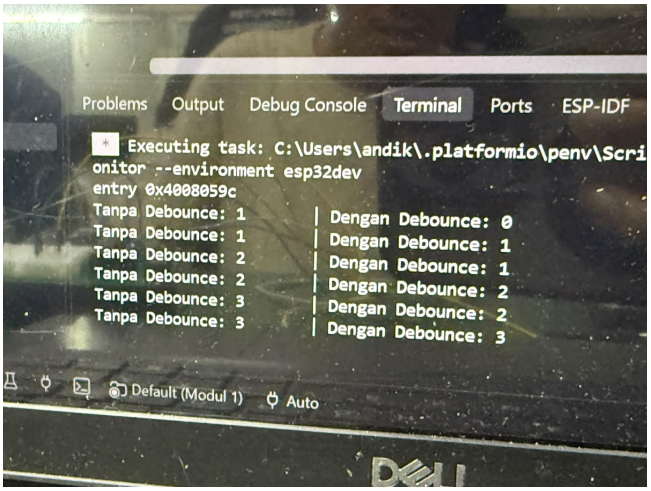
Gambar 7: Rangkaian Percobaan 3a.



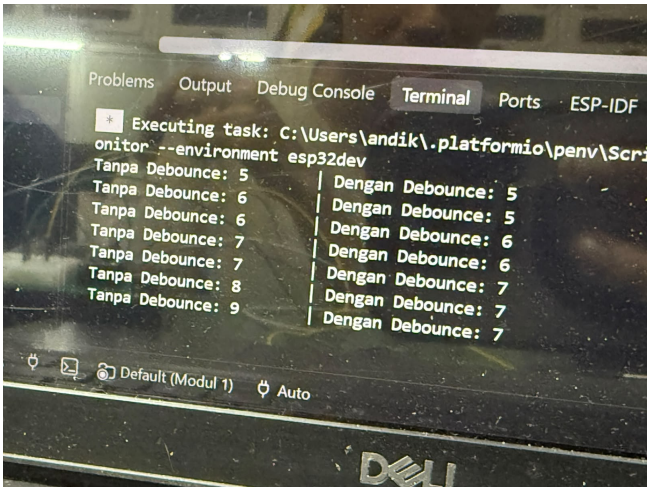
Gambar 8: Percobaan 3b - State 0.



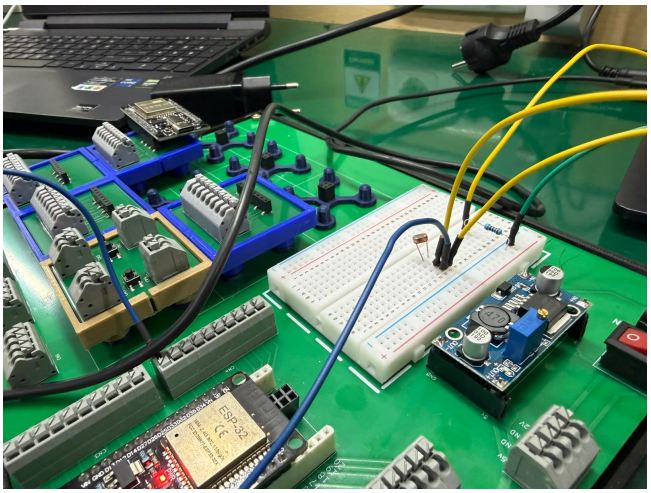
Gambar 9: Percobaan 3b - State 1.



Gambar 10: Pengujian Percobaan 4 - Ditekan Lambat.



Gambar 11: Pengujian Percobaan 4 - Ditekan Cepat.



Gambar 12: Rangkaian Percobaan 5.